FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES

CCS007L
(COMPUTER PROGRAMMING 2)

EXERCISE
8

INTRODUCTION TO OBJECT ORIENTED PROGRAMMING

Student Name / Group Name:	Jed L. Hizon
Members (if Group):	**Name** / **Role**
Section:	AN11 – Term 2
Professor:	Mr. Cezar

I. PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Analyze a complex problem and identify and define the computing requirements appropriate to its solution. [PO: B]

II. COURSE LEARNING OUTCOME/S (CLO)ADDRESSED BY THE LABORATORY EXERCISE

- Understand the fundamental principles of C-string manipulations, pointer and memory allocation and structures using C++ [CLO: 1]

III. INTENDED LEARNING OUTCOME/S (ILO) OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- Apply the knowledge of object oriented programming in writing C++ programs.
- Create record management with OOP application

IV. BACKGROUND INFORMATION

Object-oriented programming (OOP) provides us with the ability to design an “object”, which ties together both properties and behaviors into a self-contained, reusable package. This allows programs to be written in a more modular fashion, which makes them easier to write and understand, and also provides a higher degree of code-reusability. Objects also provide a more intuitive way to work with our data by allowing us to define how we interact with the objects, and how they interact with other objects. Note that OOP doesn’t replace traditional programming methods. Rather, it gives you additional tools in your programming tool belt to manage complexity when needed.

While C++ provides a number of fundamental data typesthat are often sufficient for solving relatively simple problems, it can be difficult to solve complex problems using just these types. One of C++’s more useful features is the ability to define your own data types that better correspond to the problem being solved. You have already seen how enumerated types and structs can be used to create your own custom data types.

V. GRADING SYSTEM / RUBRIC

Trait	(Excellent)	(Good)	(Fair)	(Poor)
Requirement Specification(30pts)	Able to identify correctly all input and output and provide alternative. **(28-20pts)**	Able to identify correctly all input and output **(25-17pts)**	Able to identify only one input or output **(22-14pts)**	Unable to identify any input and output **(20-11pts)**
Data type(20pts)	Able to apply required data type or data structure and produce correct results **(18-20pts)**	Able to apply required data type or data structure and produce partially correct results **(15-17pts)**	Able to identify required data type or data structure but does apply correctly **(12-14pts)**	Unable to identify required data type **(9-11pts)**
Input Validation(20pts)	The program works and meets all specifications. Does exception al checking for errors and out-of- range data **(18-20pts)**	The program works and meets all specifications. Does some checking for errors and out of range data **(15-17pts)**	The program produces correct results but does not display correctly Does not check for errors and out of range data **(12-14pts)**	The program produce s incorrect results **(9-11pts)**
Free from syntax, logic, and runtime errors (10pts)	Unable to run program **(10pts)**	Able to run program but have logic error **(8-9pts)**	Able to run program correctly without any logic error and display inappropriate output **(6-7pts)**	Able to run program correctly without any logic error and display appropriate output **(5pts)**
Delivery (10pts)	The program was delivered on time **(10pts)**	The program was delivered after 5 minutes from the time required. **(8-9pts)**	The program was delivered after 10 minutes from the time required. **(6-7pts)**	The program was delivered after 15 (or more) minutes from the time required. **(5pts)**
Use of Comments (10pts)	Specific purpose is noted for each function, control structure, input requirements, and output results. **(10pts)**	Specific purpose is noted for each function and control structure. **(8-9pts)**	Purpose is noted for each function. **(6-7pts)**	No comments included. **(5pts)**

VI. LABORATORY ACTIVITY

INSTRUCTIONS:
Copy your source codes to be pasted in this document as well as a screen shot of your running output.

ACTIVITY 8.1: Getting student grades

Create a program that will accept student id number, name, and grades for three subjects then display the average of the grades together with the other information.

EXAMPLE PROGRAM OUTPUT:

```
Enter admission number: 2018-02799
Enter student name: Joseph Q. Calleja
Enter grades in English, Math, Science:
91
90
92
Admission number: 2018-02799
Student name: Joseph Q. Calleja
English: 91
Math: 90
Science: 92
Average: 91

Press any key to continue . . .
```

```
Enter admission number: 1054-324
Enter student name: Jed
Enter grades in English, Math, Science:
91
94
56
Admission number: 1054-324
Student name: Jed
English: 91
Math: 94
Science: 56

--------------------------------
Process exited after 15.4 seconds with return value 0
Press any key to continue . . .
```

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Student{
    string admission;
    string name;
    string subject[3] = {"English", "Math", "Science"};
    string grade[3];
};

int main(){
    Student student;

    cout << "Enter admission number: ";
    cin >> student.admission;
    cout << "Enter student name: ";
    cin >> student.name;

    cout << "Enter grades in English, Math, Science:" << endl;
    for(int i = 0; i < 3; i++){
        cin >> student.grade[i];
    }

    cout << "Admission number: " << student.admission << endl;
    cout << "Student name: " << student.name << endl;

    for(int i = 0; i < 3; i++){
        cout << student.subject[i] << ": ";
        cout << student.grade[i] << endl;
    }
    return 0;
}
```

ACTIVITY 8.2: Student records

Redo your program in Activity 8.1, this time your program must be able to accept up to 5 student records. Your program may use structures, arrays, pointers, or linked lists to store data. The program will accept student id number, name, and grades for three subjects then display the average of the grades together with the other information.

```
#include <iostream>
#include <string>
using namespace std;

class StudentRecords {
private:
   string subject[3] = { "English", "Math", "Science" };

   struct Student {
      string admission;
      string name;
      float grade[3];
   };

   struct Node {
      Student student;
      Node* next;
   };

   Node* head = nullptr;

   void addStudent(const Student& student) {
      Node* newNode = new Node;
      newNode->student = student;
      newNode->next = head;
      head = newNode;
   }
```

```
public:
   void addStudents() {
      Student s;
      for (int i = 0; i < 2; i++) {
         cout << "\tStudent " << i + 1 << endl;
         cout << string(30, '-') << endl;

         cout << "Enter admission number: ";
         cin >> s.admission;
         cout << "Enter student name: ";
         cin >> s.name;

         cout << "Enter grades in English, Math, Science:" << endl;

         for (int j = 0; j < 3; j++) {
            cin >> s.grade[j];
         }
         addStudent(s);
      }
   }

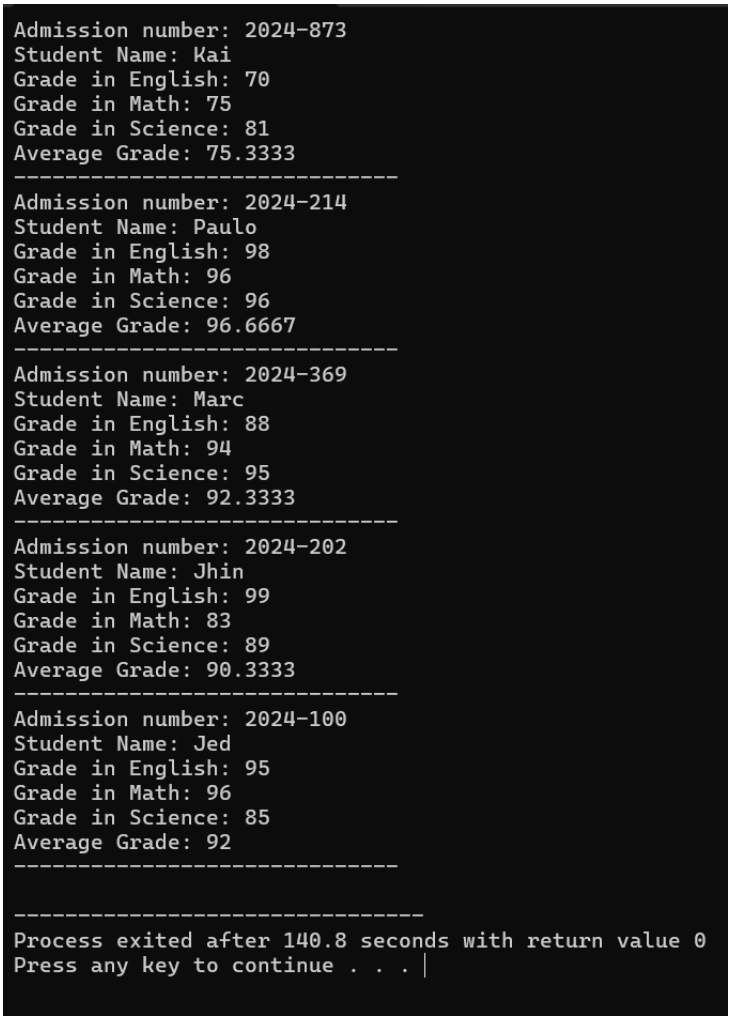
   void viewList() {
           system("cls");
      Node* current = head;
      while (current != nullptr) {
         cout << "Admission number: " << current->student.admission << endl;
         cout << "Student Name: " << current->student.name << endl;

         float sum = 0;
         for (int i = 0; i < 3; ++i) {
            cout << "Grade in " << subject[i] << ": ";
            cout << current->student.grade[i] << endl;
            sum += current->student.grade[i];
         }

         float average = sum / 3;
         cout << "Average Grade: " << average << endl;
         cout << string(30, '-') << endl;
         current = current->next;
      }
   }
};
```

```
int main() {
   StudentRecords record;
   record.addStudents();
   record.viewList();
   return 0;
}
```

```
        Student 1
------------------------------
Enter admission number: 2024-100
Enter student name: Jed
Enter grades in English, Math, Science:
95
96
85
        Student 2
------------------------------
Enter admission number: 2024-202
Enter student name: Jhin
Enter grades in English, Math, Science:
99
83
89
        Student 3
------------------------------
Enter admission number: 2024-369
Enter student name: Marc
Enter grades in English, Math, Science:
88
94
95
        Student 4
------------------------------
Enter admission number: 2024-214
Enter student name: Paulo
Enter grades in English, Math, Science:
98
96
96
        Student 5
------------------------------
Enter admission number: 2024-873
Enter student name: Kai
Enter grades in English, Math, Science:
70
75
81
```


Admission number: 2024-873
Student Name: Kai
Grade in English: 70
Grade in Math: 75
Grade in Science: 81
Average Grade: 75.3333

Admission number: 2024-214
Student Name: Paulo
Grade in English: 98
Grade in Math: 96
Grade in Science: 96
Average Grade: 96.6667

Admission number: 2024-369
Student Name: Marc
Grade in English: 88
Grade in Math: 94
Grade in Science: 95
Average Grade: 92.3333

Admission number: 2024-202
Student Name: Jhin
Grade in English: 99
Grade in Math: 83
Grade in Science: 89
Average Grade: 90.3333

Admission number: 2024-100
Student Name: Jed
Grade in English: 95
Grade in Math: 96
Grade in Science: 85
Average Grade: 92

Process exited after 140.8 seconds with return value 0
Press any key to continue . . .

VII. QUESTION AND ANSWER

Briefly answer the questions below. Avoid erasures. For group activity, specify the name of GROUP MEMBER/s who answered the question. Do not forget to include the source for all NON-ORIGINAL IDEAS.

• What is the use of a class keyword? the class keyword is utilized for declaring a fresh class, acting as a template for generating objects. This class packages data pertaining to the object and furnishes functions to manipulate this data. Moreover, it facilitates encapsulation, inheritance, and polymorphism, all of which are fundamental principles in object-oriented programming.
• How Object Oriented Programming Language does differs from Procedural (traditional) PL? Object-oriented programming (OOP) languages, such as C++, structure code around objects containing both data and behavior, enhancing modularity and reuse. Conversely, procedural programming languages center on functions or procedures manipulating data, often without encapsulation or inheritance. OOP enables concepts like inheritance and polymorphism, fostering more adaptable and modular code structures compared to procedural counterparts.

VIII. REFERENCES

- Zak, Dianne (2016). An Introduction to Programming with C++
- Deitel, Paul & Deitel, Harvey (2012). C++ How To Program, Eighth Edition
- https://www.cprogramming.com/tutorial/lesson19.html